WAB

Provadis School of International Management and Technology

Comparing Suffix Automata Against Suffix Arrays For Longest Common Substring Queries

Rubin Chempananickal James

rubin.chempananickaljames@stud-provadis-hochschule.de

Matriculation Number: D876

Department: Information Technology

Module: Algorithmen und Datenstrukturen

Reviewer: Prof. Dr. Volker Scheidemann

March 9, 2026

Abstract

The objective of this paper was to compare the performance of two algorithms, the *suffix automaton* and the *enhanced suffix array*, for solving the longest common substring problem **between two strings** in a pure Python implementation.

The algorithms were tested on multiple scenarios, from best case to worst case, and on multiple string lengths, from 100 to 10,000 characters, with multiple runs for each scenario and string length to ensure statistical significance of the results. Both the build time and the query time were measured, as well as the memory used both for building the data structure and for querying it.

The results showed that **so long as one is programming in pure Python, and so long as only two strings are involved**, an enhanced suffix array offers practically zero advantages over a suffix automaton. The suffix automaton was substantially faster to build and to query in all but one scenario, and although the enhanced suffix array had an overall smaller index size, this would do very little in real-life memory-constrained applications, as its peak memory usage was either comparable to the suffix automaton or only barely smaller in all scenarios and string lengths. Not to mention that the suffix automaton is much simpler to implement.

This is potentially due to the nature of Python, as it is a high level interpreted language where the memory management is abstracted away from the programmer, leading to even simple data types like integers and lists having a much higher memory overhead than in lower level languages. Due to Python's memory management, the enhanced suffix array would also lose its cache locality, which is the main reason for its compactness in lower level languages like C.

Abstrakt

Das Ziel dieser Arbeit war es, die Leistung von zwei Algorithmen, dem Suffix Automaton und dem Enhanced Suffix Array, bei der Lösung des Longest Common Substring-Problems ***zwischen zwei Zeichenketten (Strings)*** *in einer reinen Python-Implementierung zu vergleichen.*

Die Algorithmen wurden in verschiedenen Szenarien getestet, von besten bis zu schlechtesten Fällen, und auf verschiedenen Stringlängen, von 100 bis 10.000 Zeichen, mit mehreren Durchläufen für jedes Szenario und jede Stringlänge, um die statistische Signifikanz der Ergebnisse sicherzustellen. Sowohl die Aufbauzeit als auch die Abfragezeit wurden gemessen, ebenso wie der für den Aufbau der Datenstruktur und für die Abfrage verwendete Speicher.

Die Ergebnisse zeigten, dass ***solange man in reinem Python programmiert und nur zwei Zeichenketten beteiligt sind****, ein Enhanced Suffix Array praktisch keine Vorteile gegenüber einem Suffix Automaton bietet. Der Suffix Automaton war wesentlich schneller im Aufbau und in der Abfrage in allen bis auf einer der Szenarien, und obwohl der Enhanced Suffix Array insgesamt eine kleinere Indexgröße hatte, würde dies in realen, speicherbeschränkten Anwendungen nur sehr wenig ausmachen, da sein Spitzen-Speicherverbrauch in allen Szenarien und Stringlängen entweder vergleichbar mit dem des Suffix Automatons oder nur ein wenig kleiner war. Auch von Vorteil ist, dass der Suffix Automaton viel einfacher zu implementieren ist.*

Dies liegt vermutlich an der Natur von Python, da es eine hochstufige interpretierte Sprache ist, bei der die Speicherverwaltung von der Programmierer/in abstrahiert wird, was dazu führt, dass selbst einfache Datentypen wie Ganzzahlen und Listen einen viel höheren Speicher-Overhead haben als in kompilierten Sprachen. Aufgrund der Speicherverwaltung von Python würde der Enhanced Suffix Array auch seine Cache-Lokalität verlieren, was der Hauptgrund für seine Kompaktheit in Sprachen wie C ist.

Contents

Abstract **II**

List of Figures **VI**

List of Tables **VII**

Glossary **VIII**

Abbreviations **IX**

1 Introduction **1**
1.1 Background 1
1.1.1 Suffix Tree 1
1.1.2 Suffix Automaton 2
1.1.3 Enhanced Suffix Array 2
1.2 Motivation 3
1.3 Research Question 3

2 Methods **4**
2.1 Hardware Specifications 4
2.2 Python Environment 4
2.3 Test Data 5
2.4 Design 7

3 Results **8**
3.1 Choice of Central Tendency 8
3.2 Time 9
3.2.1 Build Time 9
3.2.2 Query Time 10
3.2.3 Total Time 10
3.3 Memory Usage 11
3.3.1 Index Size 12
3.3.2 Peak Memory Usage 12

4 Conclusion **14**
4.1 Results and Discussion 14
4.2 Limitations 15
4.3 Practical Considerations 15
4.4 Future Work 16

Bibliography iii

Appendix iv

AI Declaration viii

Declaration of Authorship xi

List of Figures

3.1 Distribution of total time for the SAM and the ESA for the random uniform scenario and string length 10000, as an example of the right-skewed nature of the data 8
3.2 Median and interquartile range (IQR) of build time for the SAM and the ESA across different scenarios and string lengths 9
3.3 Median and interquartile range (IQR) of query time for the SAM and the ESA across different scenarios and string lengths 10
3.4 Median and interquartile range (IQR) of total time for the SAM and the ESA across different scenarios and string lengths 11
3.5 Median and interquartile range (IQR) of index size for the SAM and the ESA across different scenarios and string lengths 12
3.6 Median and interquartile range (IQR) of peak memory usage for the SAM and the ESA across different scenarios and string lengths 13

List of Tables

2.1 Details of the Python environment used for implementation and testing . 5
2.2 Description of the alphabets used for generating test data 5
2.3 Description of test scenarios used for benchmarking 6

Glossary

base The building blocks of DNA and RNA, represented by the letters A, C, G, T (for DNA) and A, C, G, U (for RNA). 5

DNA Deoxyribonucleic Acid, the molecule that carries genetic information in living organisms. 1, 5

RNA Ribonucleic Acid, a molecule that is transcribed from DNA and which then gets translated into proteins. 1, 5

sentinel A unique character used to separate concatenated strings in a suffix array or suffix tree, which is lexicographically smaller than any other character in the strings. Usually characters like \0 (null character), $, or # are used as sentinels, as they are ahead of all other characters in the ASCII table. 3, 7, 16

Abbreviations

DAWG Directed Acyclic Word Graph.

ESA Enhanced Suffix Array.

IQR Interquartile Range.

LCP Longest Common Prefix.

LCS Longest Common Substring.

SA Suffix Array.

SA-IS Suffix Array with Induced Sorting.

SAM Suffix Automaton.

1 Introduction

1.1 Background

The Longest Common Substring (LCS) problem is a fundamental problem in computer science, with applications in multiple domains, bioinformatics being one of the most prominent. This is due to the fact that DNA and RNA sequences are usually encoded as strings for bioinformatics workflows, in a format called FASTA, originally described by Pearson and Lipman in 1988[1].

The LCS problem is defined as follows:

> Given two strings S and T, each of length at most n, the longest common substring (LCS) problem is to find a longest substring common to S and T.[2]

For example, given the strings "AG**CTAGC**" and "T**CTAGC**TA", the longest common substring is "CTAGC", which has a length of 5. The LCS problem can be solved using various algorithms, such as dynamic programming, suffix trees, suffix arrays, and suffix automata, each with different time and space complexities.

Of particular interest are the Suffix Automaton (SAM) (also known as Directed Acyclic Word Graph (DAWG)) and the Suffix Array (SA) with Longest Common Prefix (LCP) Array (also known as an Enhanced Suffix Array (ESA)), which both yield the search result in linear time, but differ in their construction time and space requirements.

Both of these data structures are derived from the suffix tree, the first linear time and space data structure for solving the LCS problem, which was introduced by Peter Weiner in 1973[3] as "bi-trees", although the uncompressed version he introduced is currently referred to as a *trie* in order to differentiate it from the modern definition of a suffix tree, which corresponds to the compressed version as described by McCreight in 1976[4].

1.1.1 Suffix Tree

A (compressed) suffix tree is constructed from a string S using an algorithm, originally called Algorithm M, mapping a finite string S of characters into an auxiliary index to S in the form of a digital search tree T whose paths are the suffixes of S, and whose terminal nodes correspond uniquely to positions within S[4].

As the suffix automaton and the enhanced suffix array are both further refinements of

the suffix tree, a suffix tree implementation will not be included in the comparisons done in this paper.

1.1.2 Suffix Automaton

A Suffix Automaton (SAM) is a deterministic finite automaton that represents all the substrings of a given string in a compact way.

> An efficient and compact data structure for representing a full index of a set of strings is a suffix automaton, a minimal deterministic automaton representing the set of all suffixes of a set of strings. Since a substring is a prefix of a suffix, a suffix automaton can be used to determine if a string x is a substring in time linear in its length $O(|x|)$, which is optimal.[5]

The SAM was first introduced by Blumer et al. in 1985[6], and its use in solving the LCS problem has been explored in various works, such as in the book *Text Algorithms* by Crochemore and Rytter, in chapter 6[7].

A notable feature of the suffix automaton is that it can be constructed with just one string, and in fact, the same automaton can then be reused to find the longest common substring of that string and any other string, by traversing the automaton with the second string and keeping track of the longest match found.

1.1.3 Enhanced Suffix Array

> The generic name *enhanced suffix array* stands for data structures consisting of the suffix array and additional tables.[8]

In this implementation, an Enhanced Suffix Array (ESA) consists of a Suffix Array (SA) and a Longest Common Prefix (LCP) Array, which is all that's necessary to solve the LCS problem in linear time because the LCS can be solved by finding the maximum value in the LCP array corresponding to suffixes (from the SA) that belong to different strings.

An SA is an array of integers that represent the starting positions of the suffixes of a string in lexicographical order, while the LCP array is an array of integers that represent the length of the longest common prefix between adjacent suffixes in the SA[8].

Currently, there are algorithms to construct both the SA and the LCP array in $O(m+n)$ time for two strings S and T of lengths m and n respectively, such as the Suffix Array with Induced Sorting (SA-IS) algorithm for the SA[9] and Kasai's algorithm for the LCP

array[10].

A notable feature of the ESA is that it can be constructed with more than two strings at once, by concatenating the strings together with unique delimiters (sentinels), and then constructing the ESA for the concatenated string. This allows for finding the LCS of all the strings at once.

1.2 Motivation

Although there have been many papers comparing either the SAM or the ESA to the suffix tree[5,6,7,8,11], there is a lack of direct comparisons between these two data structures, even though they both fundamentally solve the same problem using the same underlying principle of being a navigable compressed representation of all suffixes of a string.

Moreover, there is a lack of concrete practical comparisons between these two structures in the context of specific programming languages, even though the obtuse memory management of languages like Python[12] could potentially lead to real-world results not matching up with theoretical expectations, especially when it comes to space usage.

1.3 Research Question

The primary research question of this paper is:

> How do the SAM and the ESA compare in terms of time and space complexity when solving the longest common substring problem **between two strings** in an everyday programming context?

Python[12] was chosen as the programming language for this paper due to the fact that it is by far the most popular programming language in the field of bioinformatics[13].

2 Methods

2.1 Hardware Specifications

The tests were run on a laptop with the following specifications:

- CPU Model: Intel Core i5-10210U
- CPU Cores: 4[14]
- CPU Threads: 8[14]
- CPU Base Frequency: 1.60 GHz[14]
- CPU Max Turbo Frequency: 4.20 GHz[14]
- RAM: 16 GB DDR4 2933 MT/s
- Storage: 512 GB NVMe SSD

In order to ensure minimal interference, the tests were run with no other applications (other than the IDE itself) running, and with the laptop plugged in and heavy background processes like Windows Update disabled.

2.2 Python Environment

Here are the details of the Python environment used for the implementation and testing of the algorithms:

Python Version	3.13.12
Operating System	Windows 11 Pro 64-bit (version 25H2)
IDE	Visual Studio Code (version 1.109.5)
Environment Manager	Miniforge (conda version 26.1.0)
Packages Used	**Package** / **Version** time / built-in tracemalloc / built-in sys / built-in matplotlib / 3.10.8 pandas / 3.0.1 jupyter / 1.1.1 Jinja2 / 3.1.6

Table 2.1: Details of the Python environment used for implementation and testing

2.3 Test Data

To ensure a comprehensive evaluation of the algorithms, synthetic test data was generated using three different alphabets and five different scenarios, each designed to cover a wide range of cases that could be encountered in real-world applications, with a particular focus on the field of bioinformatics, where the LCS problem is particularly relevant due to the nature of DNA and RNA sequences being represented as strings.

The alphabets used for the test data were as follows:

Alphabet Name	**Description**	**Characters**
Full	Every lowercase letter	a, b, c, ..., z
DNA	The four DNA bases	A, C, G, T
Disjoint	Two disjoint sets of the lowercase letters	String 1: a, b, c, ..., m String 2: n, o, p, ..., z

Table 2.2: Description of the alphabets used for generating test data

Five separate scenarios were tested, in order to cover as wide a range of cases as possible:

Scenario Name	**Description**	**Alphabet**
random_uniform	Both strings are generated randomly with a uniform distribution of characters	Full
mutated_implant	A common motif is implanted in both strings with mutations	DNA
repetitive_with_noise	Strings are formed by repeating a pattern with some mutations	DNA
near_identical	Strings are nearly identical, with minor mutations preferentially at the edges	Full
disjoint_alphabet	Strings are generated from completely disjoint alphabets	Disjoint

Table 2.3: Description of test scenarios used for benchmarking

The justifications for the choice of these scenarios are:

- The *random_uniform* scenario serves as a baseline for the performance of the algorithms on completely random data.
- The *mutated_implant* scenario simulates a common real-world case in bioinformatics, where two otherwise unrelated long sequences share a relatively short common motif between them at unrelated positions[15,16]. It's basically the "needle in a haystack" scenario.
- The *repetitive_with_noise* scenario simulates another common case in bioinformatics, where sequences are formed by repeating a pattern (tandem repeats[17]) with some mutations, which can lead to multiple short LCSs.
- The *near_identical* scenario tests the performance of the algorithms on strings that are nearly identical, so that there would be one very long LCS which would have to be held in memory.
- The *disjoint_alphabet* scenario tests the case where the two strings have no common characters, which should result in an LCS of length 0. The algorithms would have to traverse the entire data structure to determine this. It also serves as a sanity check to ensure that the algorithms are correctly identifying the lack of common substrings.

2.4 Design

The algorithms themselves were implemented in pure Python (version 3.13.12), without the aid of any external libraries (although external libraries were used for gathering the results and plotting the graphs).

The SAM was implemented using the standard on-line construction algorithm[18]. To find the LCS of the first string and the second string, the SAM was created with the first string, and then traversed through by the second string, keeping track of the longest match found.

To give the Enhanced Suffix Array (ESA) the best chance possible, the SA was constructed using the Suffix Array with Induced Sorting (SA-IS) algorithm, the current state-of-the-art algorithm for constructing it in linear time[9]. The SA was constructed for the concatenated string of the two strings being compared, with a sentinel character in between them (`\0` in this case) to separate them. The LCP array was constructed using Kasai's algorithm[10], which is a linear time algorithm for constructing it from the SA. The SA and the LCP array were then combined to form the ESA.

For each scenario, **25 string pairs** were generated at each of the lengths 100, 300, 600, 1000, 2000, 3000, 4500, 6000, 8000, and 10000 characters. Each string pair was benchmarked on both algorithms **20 times** to try and ensure statistical significance of the results. A fixed random seed of **42** was used throughout to ensure reproducibility.

The testing code was designed to run both algorithms on the same string pairs, and to measure both the time taken and the peak memory usage for each run. The time taken was measured using the `time` module, while the peak memory usage was measured using the `tracemalloc` module, which is a built-in Python module for tracking memory allocations. The index size for each algorithm was computed using the `sys.getsizeof` function, which returns the size of an object by recursively summing the sizes of all referenced objects.

The full testing code, including a requirements file and instructions for running it, is provided in the supplementary materials. A complete list of generated string pairs with their corresponding LCS values, as well as the raw test results, are also included. Additionally, the code used for plotting the results (using the `matplotlib` library) is provided.

3 Results

3.1 Choice of Central Tendency

The data collected seems to be heavily right-skewed, with some outliers. This is irrespective of the scenario, string length, algorithm, or even the type of measurement (build time, query time, total time, or memory usage). This is likely due to cache misses, other background processes running on the computer, or any number of other random factors.

Therefore, the **median** was chosen as the measure of central tendency for the results, as the data is not normally distributed and the median is more robust to outliers than the mean. Consequently, the interquartile range (IQR) was chosen as the measure of variability.

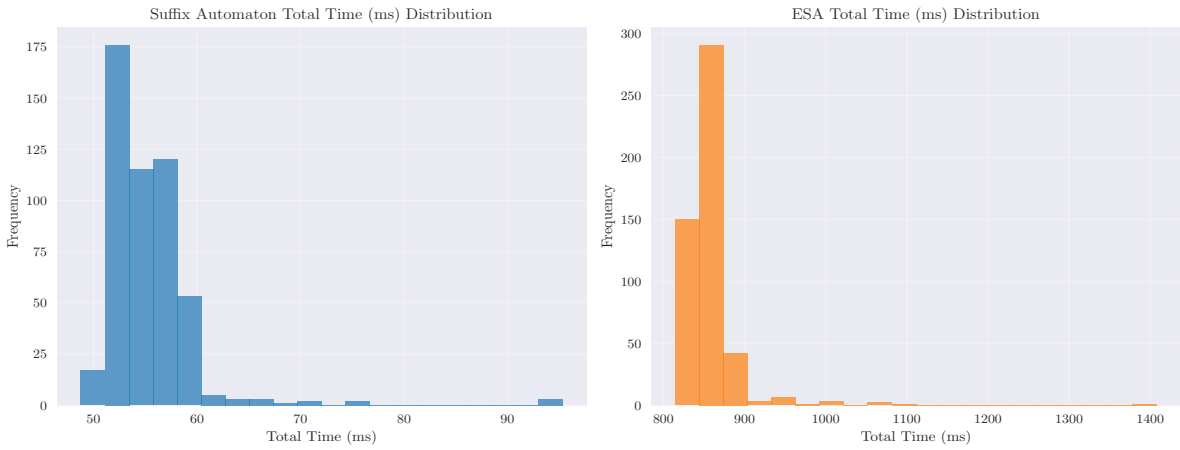

Figure 3.1: Distribution of total time for the SAM and the ESA for the random uniform scenario and string length 10000, as an example of the right-skewed nature of the data

Tables with the exact values of the median and IQR for all measurements are included as a CSV file in the supplementary materials (`Code/Results/summary_stats.csv`), which includes other statistics such as the mean and standard deviation.

Since both algorithms are expected to scale linearly with input length (O(n) time and space complexity), a linear regression was fitted to the median values of each measurement to estimate the empirical scaling constant (slope) per algorithm, scenario, and metric. The goodness of fit of each regression was also measured using the coefficient of determination (R^2), which was found to be very close to 1 in every case. The ratio of the SAM slope to the ESA slope was then calculated as a hardware-independent measure of relative

performance, as the absolute slope values are likely machine-dependent. Some ratios are cited in the text where relevant and full tables are included in the appendix (see page iv) and as a CSV file in the supplementary materials (`Code/Results/slope_ratios.csv`).

3.2 Time

The time taken to construct the SAM and the ESA and to find the LCS was measured for each scenario and string length, and the results are shown in the following plots.

The build time, query time, and total time are shown separately to give a more detailed view of the performance of the algorithms. The median is shown as a line, and the IQR is shown as a shaded area around the line. Because the IQR is very small for most of the data points, it may not be visible in the plots.

3.2.1 Build Time

The build time is the time taken to construct the data structure (the SAM or the ESA) from the string(s).

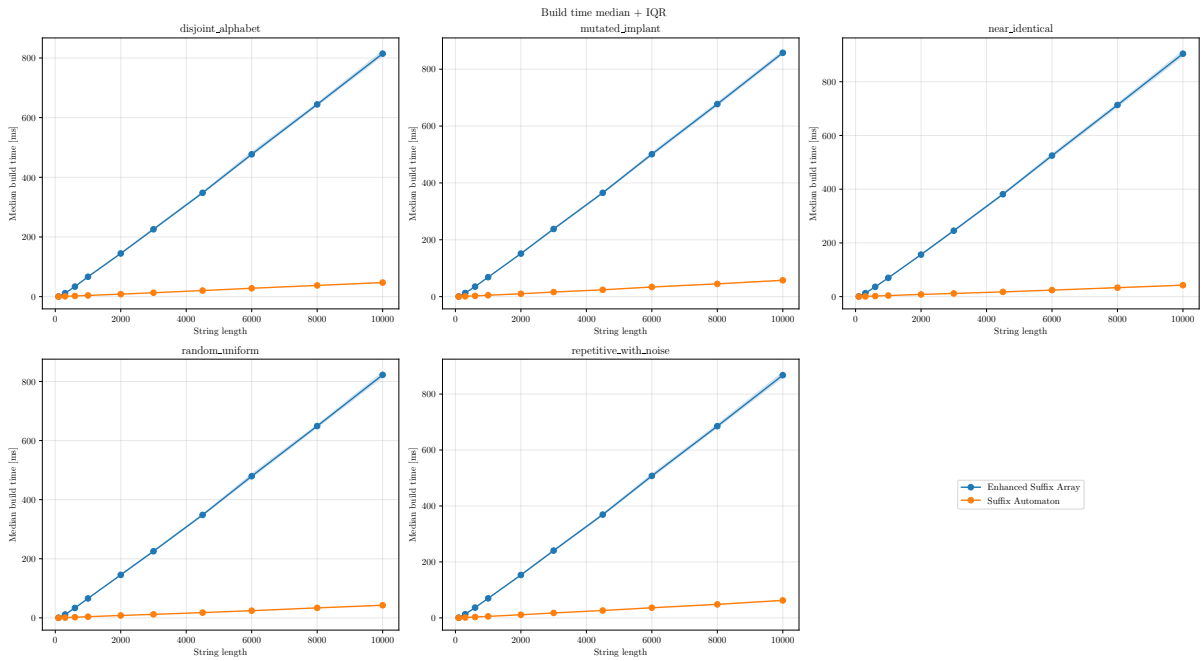

Figure 3.2: Median and interquartile range (IQR) of build time for the SAM and the ESA across different scenarios and string lengths

As can be seen from the plot, the build times for both algorithms tend to follow a linear trend with respect to the string length (as is to be expected), but the ESA has a much higher build time than the SAM, especially as the string length increases.

3.2.2 Query Time

The query time is the time taken to find the LCS of the two strings using the constructed data structure.

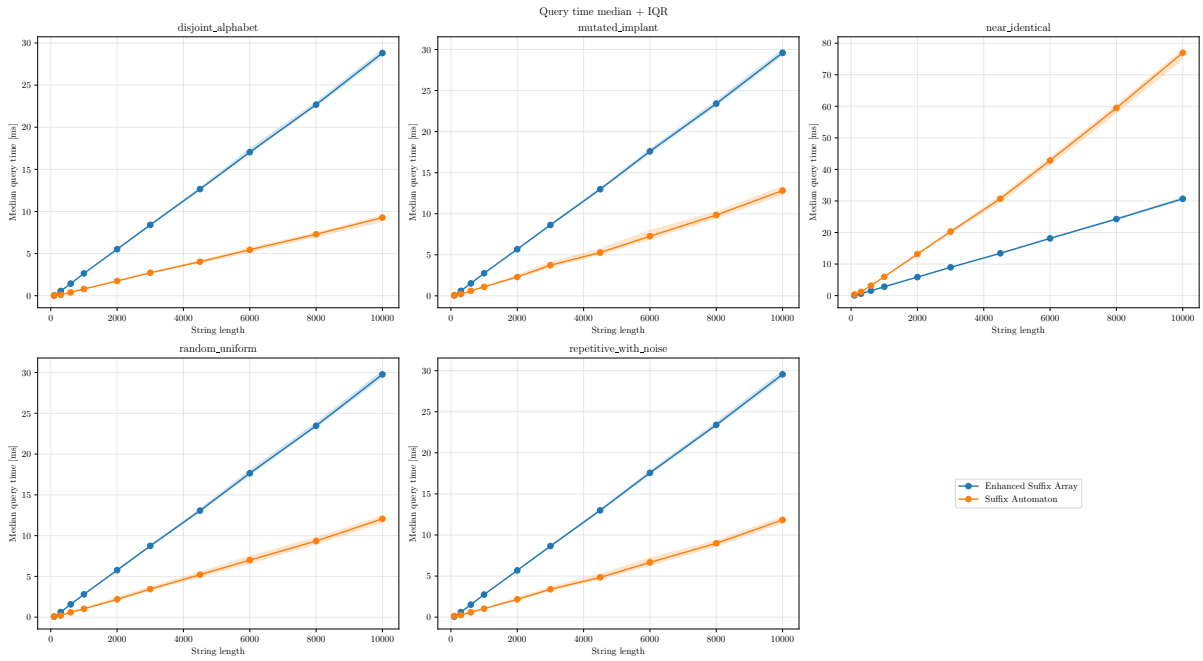

Figure 3.3: Median and interquartile range (IQR) of query time for the SAM and the ESA across different scenarios and string lengths

The query time for both algorithms also tends to follow a linear trend with respect to the string length, and for most cases, the SAM has a shorter query time than the ESA, with near_identical being the only exception. This outlier can be explained by the fact that in the near_identical scenario, where the LCS is singular and very long, causing the SAM to have to traverse a long path in the automaton, whereas the ESA can find the LCS by directly inspecting the LCP between the relevant suffixes, and that too, only once. This leads to the interesting observation that *the query time of the SAM grows in proportion to the length of the LCS*, at least in Python.

Also note that the query time for both algorithms is extremely small compared to the build time, and even with string pairs that are each 10,000 characters long, the median query time remained under 0.1 seconds.

3.2.3 Total Time

The total time represents the time from the start of the construction of the data structure to the end of finding the LCS, so it is essentially the sum of the build time and the query time.

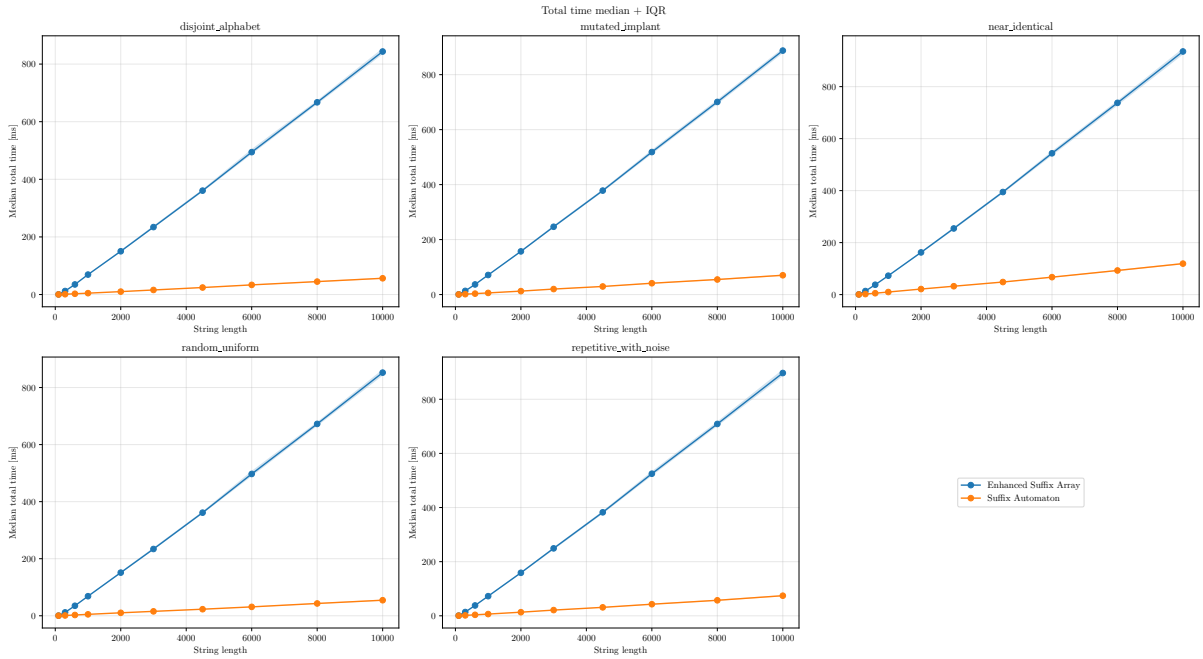

Figure 3.4: Median and interquartile range (IQR) of total time for the SAM and the ESA across different scenarios and string lengths

Because the build time is so high for the ESA compared to the SAM, the total time for the ESA is also much higher than that of the SAM across all scenarios and string lengths, with the difference becoming more pronounced as the string length increases.

In the worst case scenario (random_uniform), the total time for the ESA is about 15.83 times higher than that of the SAM (SAM/ESA slope ratio of 0.07). And even in the best case scenario (near_identical), the ESA is about 7.96 times higher than that of the SAM (SAM/ESA slope ratio of 0.13).

This is in spite of the fact that the ESA was built with the SA-IS and Kasai's algorithms, avoiding the traditional naive algorithms which would have used Python's built-in sorting, which would have rendered the time complexity to be $O(n \log n)$ for a combined string of length n.

3.3 Memory Usage

The memory usage was measured for both the build phase and the query phase, as well as the total memory usage, which is the peak memory usage during the entire process. The index size was also measured, which is the size of the data structure (the SAM or the ESA) in memory.

3.3.1 Index Size

The index size was measured using the `sys.getsizeof` function, which returns the size of an object in bytes, including the size of all referenced objects recursively.

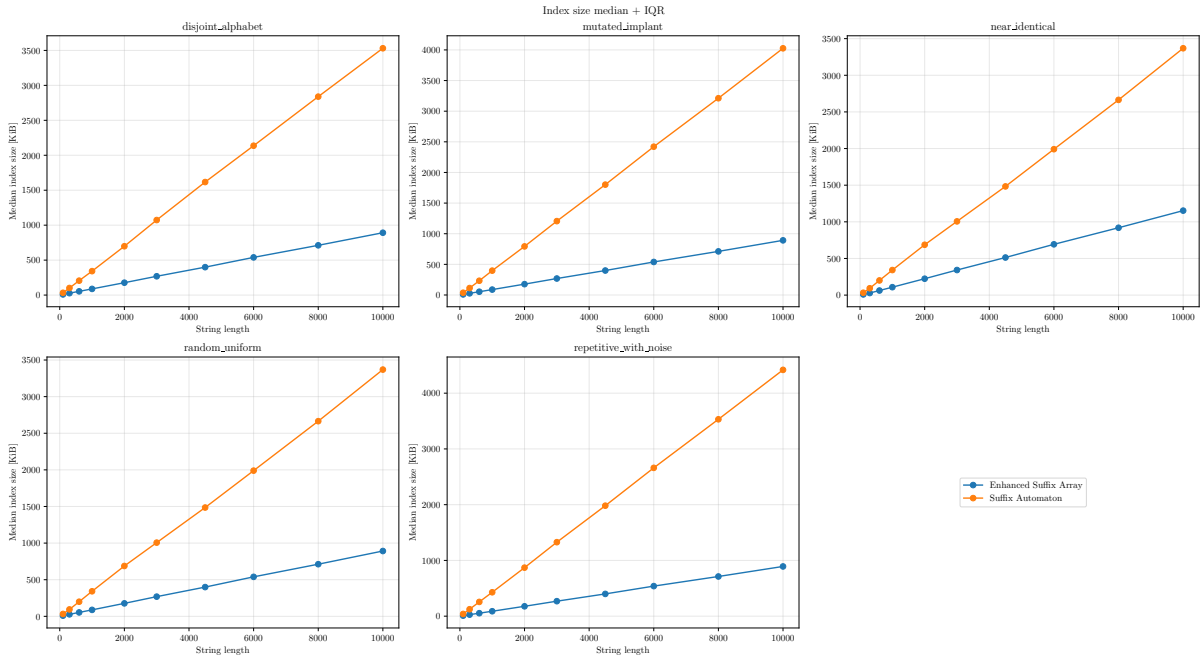

Figure 3.5: Median and interquartile range (IQR) of index size for the SAM and the ESA across different scenarios and string lengths

As can be seen from the plot, the index size for the SAM is much larger than that of the ESA.

In the best case scenario (near_identical), the SAM has an index size that is about 2.89 times larger than that of the ESA (SAM/ESA slope ratio of 2.89), and in the worst case scenario (repetitive_with_noise), the SAM has an index size that is about 4.97 times larger than that of the ESA (SAM/ESA slope ratio of 4.97).

This is to be expected, as the SAM is a much more complex data structure, being a deterministic finite automaton, while the ESA is essentially just two arrays of integers.

3.3.2 Peak Memory Usage

The peak memory usage is measured using the `tracemalloc` module, which tracks memory allocations in Python.

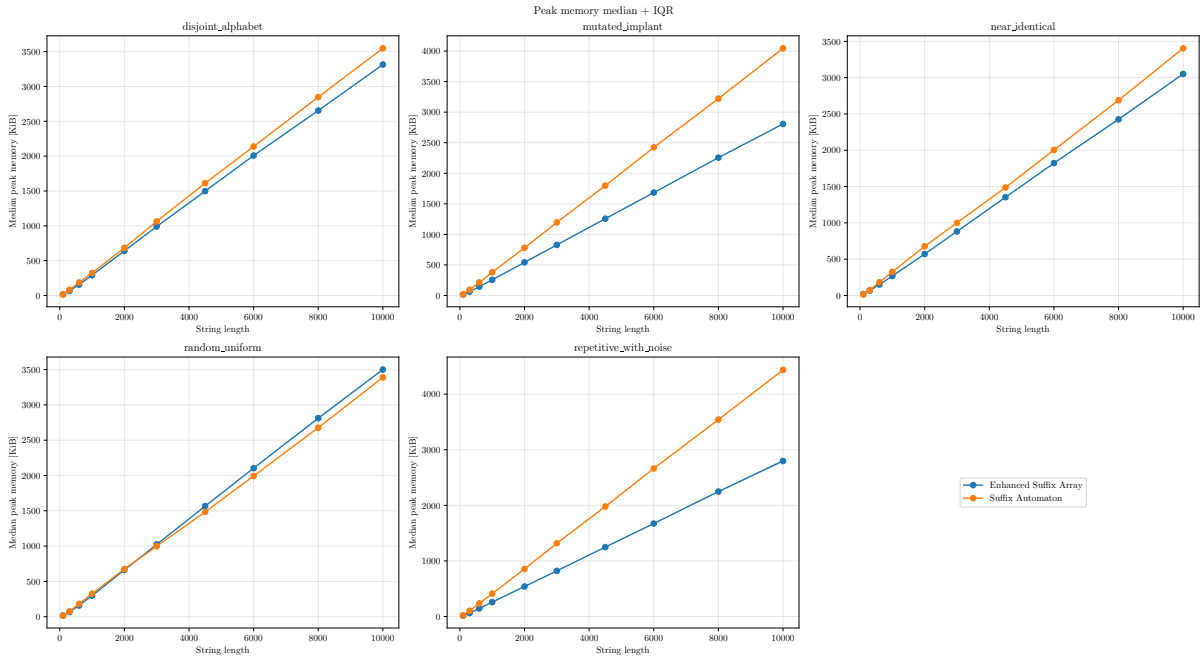

Figure 3.6: Median and interquartile range (IQR) of peak memory usage for the SAM and the ESA across different scenarios and string lengths

This result was considerably more surprising. As the plot shows, peak memory usage is comparable between the two algorithms in three out of five scenarios, with the ESA even exhibiting 5% higher peak memory usage than the SAM in the random_uniform scenario.

In the two remaining scenarios where the SAM consumes more peak memory (mutated_implant and repetitive_with_noise), the difference is far less pronounced than what the index size differences would have led one to expect. The SAM had about 43% higher peak memory usage than the ESA in the mutated_implant scenario (slope ratio of 1.43), and about 58% higher in the repetitive_with_noise scenario (slope ratio of 1.58).

This lends credence to the argument that a pure Python implementation of the ESA offers no decisive advantage in truly memory-constrained environments, where peak memory usage (and not final index size) determines whether the algorithm can even run on the available hardware in the first place.

4 Conclusion

4.1 Results and Discussion

In this paper, two algorithms for solving the longest common substring (LCS) problem between two strings were compared: the Suffix Automaton (SAM) and the Enhanced Suffix Array (ESA). The algorithms were implemented in pure Python and tested on a variety of scenarios and string lengths, with the results measured in terms of build time, query time, total time, peak memory usage, and index size.

The results showed that the SAM generally outperformed the ESA in terms of build time and total time, while the query time was more variable depending on the scenario. The SAM had a shorter query time than the ESA in most cases, except for the near_identical scenario where the ESA had a shorter query time due to the nature of the LCS being singular and very long.

In terms of memory usage, while SAM had a much larger index size than the ESA, with the SAM reaching above 4 MiB median for strings of length 10,000, (compared to the ESA which stayed barely above 1 MiB median for the same string length), the peak memory usage of the SAM and the ESA were more comparable in most scenarios, negating any potential benefit of the ESA in memory-constrained environments.

The SAM's performance advantage in build time can be attributed to its more straightforward construction algorithm, which processes the string character by character while maintaining a relatively simple state structure. In contrast, the ESA construction involves first building the suffix array using the SA-IS algorithm, followed by computing the LCP array using Kasai's algorithm, introducing additional computational overhead despite both algorithms being theoretically linear.

The memory results revealed an interesting discrepancy between theoretical expectations and practical outcomes. Despite the ESA having a significantly smaller index size (barely above 1 MiB for 10,000-character strings compared to the SAM's 4+ MiB), the peak memory usage during construction and query remained comparable between the two approaches. This can be attributed to Python's memory management system and its substantial per-object overhead. As a dynamically typed, interpreted language, Python allocates memory for objects with considerable metadata, and the temporary data structures created during algorithm execution consume memory that is not reflected in the final index size measurements. This Python-specific behavior suggests that the ESA's theoretical space advantage may be diminished in practice when using high-level languages

with significant runtime overhead.

Another noteworthy observation was that the query time of the SAM appeared to grow proportionally to the length of the resulting LCS. This behavior, particularly evident in the near_identical scenario where the ESA outperformed the SAM, suggests that the SAM's query performance is influenced not only by the input string length but also by the characteristics of the match itself. This could potentially be due to the way the Python implementation constructs the result set of matching substrings, although it warrants further investigation to determine whether this pattern persists in lower-level language implementations or with different result collection strategies.

In summary, to answer the original research question of which algorithm is more efficient for finding the longest common substring between two strings in Python, the ***Suffix Automaton (SAM)*** *proves to be the more favorable choice given the criteria examined in this study.*

4.2 Limitations

It is important to note that this study focused specifically on the LCS problem **between exactly two strings** in a Python implementation. The results and conclusions are therefore most applicable to this particular use case and programming environment.

The choice of Python as the implementation language, while justified by its prevalence in bioinformatics, introduces language-specific characteristics that may not generalize to implementations in lower-level languages such as C[19] or Rust[20], where memory management is more explicit and overhead is significantly reduced. Additionally, the benchmark scenarios, while diverse, represent a finite set of test cases with a focus on bioinformatics-relevant patterns, and may not capture the full spectrum of possible input characteristics that could influence algorithm performance.

Furthermore, this comparison examined relatively straightforward implementations of both algorithms without exploring potential optimizations such as specialized memory allocators.

4.3 Practical Considerations

While the benchmarks demonstrated the SAM's overall advantage for the specific two-string LCS problem, the choice between these algorithms in practice should be guided by the particular use case at hand.

If multiple strings need to be compared against the same "reference" string, the SAM

becomes significantly more attractive. The automaton needs to be constructed only once for the reference string and can then be reused for all subsequent comparisons, making the construction time effectively "amortized" across all queries. This makes the SAM particularly well-suited for applications such as pattern matching in genomic databases, where a single reference genome is queried repeatedly against many sample sequences.

Conversely, if the LCS of more than two strings needs to be found simultaneously, the ESA still holds a distinct advantage. The ESA can be constructed for an arbitrary number of strings by concatenating them with unique sentinel characters, so long as one can find enough sentinels that are lexicographically smaller than any character in the input strings and not present in the strings themselves. The equivalent functionality for the SAM, the "Generalized Suffix Automaton"[5] - is a substantially more complex data structure that maintains the same $O(|\sum_{i=1}^{k} n_i|)$ construction complexity as the ESA, where n_i is the length of string i and k is the number of strings. Given the increased implementation complexity without a corresponding performance benefit, the ESA would likely be preferable for multi-string LCS problems, unless a "sentinel-free" approach is specifically needed.

4.4 Future Work

Due to the narrow scope of this paper, more advanced algorithms like the FM-Index[21] were not included in the comparisons, but it could be interesting to compare against in future work, especially given that it is built on top of the ESA idea and optimized for using minimal space.

Additionally, a comparison between these algorithms when they are written in Cython[22] would be worth further exploration, as many performance critical Python packages are implemented in Cython to achieve a balance between Python's ease of use and C's efficiency.

Bibliography

[1] William R Pearson and David J Lipman. „Improved tools for biological sequence comparison.“ In: *Proceedings of the National Academy of Sciences* 85.8 (1988), pp. 2444–2448.

[2] Amihood Amir et al. „Dynamic and internal longest common substring“. In: *Algorithmica* 82.12 (2020), pp. 3707–3743.

[3] Peter Weiner. „Linear pattern matching algorithms“. In: *14th Annual Symposium on Switching and Automata Theory (swat 1973)*. 1973, pp. 1–11. DOI: 10.1109/SWAT.1973.13.

[4] Edward M. McCreight. „A Space-Economical Suffix Tree Construction Algorithm“. In: *J. ACM* 23.2 (Apr. 1976), pp. 262–272. ISSN: 0004-5411. DOI: 10.1145/321941.321946. URL: https://doi.org/10.1145/321941.321946.

[5] Mehryar Mohri, Pedro Moreno, and Eugene Weinstein. „General suffix automaton construction algorithm and space bounds“. In: *Theoretical Computer Science* 410.37 (2009). Implementation and Application of Automata (CIAA 2007), pp. 3553–3562. ISSN: 0304-3975. DOI: https://doi.org/10.1016/j.tcs.2009.03.034. URL: https://www.sciencedirect.com/science/article/pii/S0304397509002370.

[6] A. Blumer et al. „The smallest automation recognizing the subwords of a text“. In: *Theoretical Computer Science* 40 (1985). Eleventh International Colloquium on Automata, Languages and Programming, pp. 31–55. ISSN: 0304-3975. DOI: https://doi.org/10.1016/0304-3975(85)90157-4. URL: https://www.sciencedirect.com/science/article/pii/0304397585901574.

[7] Maxime Crochemore and Wojciech Rytter. *Text algorithms*. Maxime Crochemore, 1994.

[8] Mohamed Ibrahim Abouelhoda, Stefan Kurtz, and Enno Ohlebusch. „Replacing suffix trees with enhanced suffix arrays“. In: *Journal of Discrete Algorithms* 2.1 (2004). The 9th International Symposium on String Processing and Information Retrieval, pp. 53–86. ISSN: 1570-8667. DOI: https://doi.org/10.1016/S1570-8667(03)00065-0. URL: https://www.sciencedirect.com/science/article/pii/S1570866703000650.

[9] Ge Nong, Sen Zhang, and Wai Hong Chan. „Two Efficient Algorithms for Linear Time Suffix Array Construction“. In: *IEEE Transactions on Computers* 60.10 (2011), pp. 1471–1484. DOI: 10.1109/TC.2010.188.

[10] Toru Kasai et al. „Linear-time longest-common-prefix computation in suffix arrays and its applications“. English. In: *Combinatorial Pattern Matching - 12th Annual Symposium, CPM 2001, Proceedings*. Ed. by Amihood Amir et al. Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics). Publisher Copyright: © Springer-Verlag Berlin Heidelberg 2001.; 12th Annual Symposium on Combinatorial Pattern Matching, CPM 2001 ; Conference date: 01-07-2001 Through 04-07-2001. Springer Verlag, 2001, pp. 181–192. ISBN: 3540422714. DOI: 10.1007/3-540-48194-x_17.

[11] Udi Manber and Gene Myers. „Suffix arrays: a new method for on-line string searches“. In: *Proceedings of the First Annual ACM-SIAM Symposium on Discrete Algorithms*. SODA '90. San Francisco, California, USA: Society for Industrial and Applied Mathematics, 1990, pp. 319–327. ISBN: 0898712513.

[12] Guido Van Rossum and Fred L. Drake. *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace, 2009. ISBN: 1441412697.

[13] Diego Mariano et al. „A Brief History of Bioinformatics Told by Data Visualization“. In: *Advances in Bioinformatics and Computational Biology*. Ed. by João C. Setubal and Waldeyr Mendes Silva. Cham: Springer International Publishing, 2020, pp. 235–246. ISBN: 978-3-030-65775-8. URL: https://bioinfo.dcc.ufmg.br/history/.

[14] Intel Corporation. *Intel® Core™ i5-10210U Processor (6M Cache, up to 4.20 GHz) Specifications*. 2019. URL: https://www.intel.com/content/www/us/en/products/sku/195436/intel-core-i510210u-processor-6m-cache-up-to-4-20-ghz/specifications.html (visited on 02/26/2026).

[15] Jerzy W. Jurka. „Chapter Thirty-Nine - Approaches to Identification and Analysis of Interspersed Repetitive DNA Sequences“. In: *Automated DNA Sequencing and Analysis*. Ed. by Mark D. Adams, Chris Fields, and J. Craig Venter. San Diego: Academic Press, 1994, pp. 294–298. ISBN: 978-0-08-092639-1. DOI: https://doi.org/10.1016/B978-0-08-092639-1.50043-5. URL: https://www.sciencedirect.com/science/article/pii/B9780080926391500435.

[16] Barbara McClintock. „The origin and behavior of mutable loci in maize“. In: *Proceedings of the National Academy of Sciences* 36.6 (1950), pp. 344–355. DOI: 10.1073/pnas.36.6.344. eprint: https://www.pnas.org/doi/pdf/10.1073/pnas.36.6.344. URL: https://www.pnas.org/doi/abs/10.1073/pnas.36.6.344.

[17] Jorge Duitama et al. „Large-scale analysis of tandem repeat variability in the human genome“. In: *Nucleic Acids Research* 42.9 (Mar. 2014), pp. 5728–5741. ISSN: 0305-1048. DOI: 10.1093/nar/gku212. eprint: https://academic.oup.com/nar/article-pdf/42/9/5728/25346735/gku212.pdf. URL: https://doi.org/10.1093/nar/gku212.

[18] A. Blumer et al. „Building the minimal DFA for the set of all subwords of a word on-line in linear time“. In: *Automata, Languages and Programming*. Ed. by Jan Paredaens. Berlin, Heidelberg: Springer Berlin Heidelberg, 1984, pp. 109–118. ISBN: 978-3-540-38886-9.

[19] Brian W Kernighan and Dennis M Ritchie. *The C programming language*. 2006.

[20] Nicholas D. Matsakis and Felix S. Klock. „The rust language“. In: *Ada Lett.* 34.3 (Oct. 2014), pp. 103–104. ISSN: 1094-3641. DOI: 10.1145/2692956.2663188. URL: https://doi.org/10.1145/2692956.2663188.

[21] P. Ferragina and G. Manzini. „Opportunistic data structures with applications“. In: *Proceedings 41st Annual Symposium on Foundations of Computer Science*. 2000, pp. 390–398. DOI: 10.1109/SFCS.2000.892127.

[22] Stefan Behnel et al. „Cython: The Best of Both Worlds“. In: *Computing in Science and Engg.* 13.2 (Mar. 2011), 31–39. ISSN: 1521-9615. DOI: 10.1109/MCSE.2010.118. URL: https://doi.org/10.1109/MCSE.2010.118.

Appendix

The full summary statistics (mean, standard deviation, median, IQR, minimum, maximum, quartiles, and sample counts) per scenario, algorithm, and metric are available in the supplementary materials at `Code/Results/summary_stats.csv`.

The full raw benchmark data can be found at `Code/Results/raw_runs.csv`, and the corresponding string pairs with their LCS values are stored in `Code/Results/case_lcs_values.csv`.

Slope Ratios

The following tables present the slope ratios between the SAM and the ESA for each scenario and metric, along with the coefficient of determination (R^2) as a measure of goodness of fit for the linear regression.

- **SAM slope** and **ESA slope**: Fitted slope of the linear regression of the median measurement against string length for the respective algorithm. The slope is to be interpreted as the constant a in the linear model $y = a \cdot x + b$, where y is the median measurement (e.g., build time) and x is the length of one of the strings in the string pair.
- **SAM / ESA ratio**: Ratio between the two slopes. Values below 1 mean that the SAM grows more slowly than the ESA, while values above 1 mean the opposite.
- **SAM** R^2 and **ESA** R^2: A measure of how closely a linear model fits the observed median values for each algorithm, with values closer to 1 indicating a stronger linear relationship.
- **Factor**: An expression of how many times larger the greater slope is compared to the smaller one, always reported as a value ≥ 1. For example, a factor of 17.30× means one algorithm's growth rate is 17.30 times steeper than the other's.
- **Worse**: The algorithm with the steeper slope. As in, the one that scales less favourably with increasing input length.

Build Time

Slope Ratios: Build Time
Linear regression slope fitted to median build time vs. string length

Scenario	SAM slope	ESA slope	SAM / ESA ratio	SAM R^2	ESA R^2	Factor	Worse
Disjoint Alphabet	0.0048	0.0825	0.0578	0.9996	0.9997	17.30×	ESA
Mutated Implant	0.0058	0.0868	0.0665	0.9993	0.9997	15.05×	ESA
Near Identical	0.0042	0.0915	0.0462	0.9989	0.9994	21.63×	ESA
Random Uniform	0.0043	0.0832	0.0511	0.9990	0.9996	19.57×	ESA
Repetitive With Noise	0.0062	0.0878	0.0709	0.9992	0.9997	14.11×	ESA

Query Time

Slope Ratios: Query Time
Linear regression slope fitted to median query time vs. string length

Scenario	SAM slope	ESA slope	SAM / ESA ratio	SAM R^2	ESA R^2	Factor	Worse
Disjoint Alphabet	0.0009	0.0029	0.3233	0.9998	0.9999	3.09×	ESA
Mutated Implant	0.0013	0.0030	0.4270	0.9986	0.9999	2.34×	ESA
Near Identical	0.0077	0.0031	2.4885	0.9979	0.9999	2.49×	SAM
Random Uniform	0.0012	0.0030	0.4036	0.9995	0.9999	2.48×	ESA
Repetitive With Noise	0.0012	0.0030	0.3913	0.9981	1.0000	2.56×	ESA

Total Time

Slope Ratios: Total Time
Linear regression slope fitted to median total time vs. string length

Scenario	SAM slope	ESA slope	SAM / ESA ratio	SAM R^2	ESA R^2	Factor	Worse
Disjoint Alphabet	0.0057	0.0854	0.0668	0.9997	0.9997	14.97×	ESA
Mutated Implant	0.0070	0.0898	0.0783	0.9994	0.9997	12.77×	ESA
Near Identical	0.0119	0.0946	0.1256	0.9985	0.9995	7.96×	ESA
Random Uniform	0.0054	0.0862	0.0632	0.9993	0.9996	15.83×	ESA
Repetitive With Noise	0.0074	0.0907	0.0814	0.9992	0.9997	12.29×	ESA

Index Size

Slope Ratios: Index Size
Linear regression slope fitted to median index size vs. string length

Scenario	SAM slope	ESA slope	SAM / ESA ratio	SAM R^2	ESA R^2	Factor	Worse
Disjoint Alphabet	0.3550	0.0892	3.9794	0.9999	1.0000	3.98×	SAM
Mutated Implant	0.4031	0.0892	4.5193	1.0000	1.0000	4.52×	SAM
Near Identical	0.3347	0.1159	2.8887	0.9998	1.0000	2.89×	SAM
Random Uniform	0.3347	0.0892	3.7522	0.9998	1.0000	3.75×	SAM
Repetitive With Noise	0.4430	0.0892	4.9661	1.0000	1.0000	4.97×	SAM

Peak Memory Usage

Slope Ratios: Peak Memory
Linear regression slope fitted to median peak memory vs. string length

Scenario	SAM slope	ESA slope	SAM / ESA ratio	SAM R^2	ESA R^2	Factor	Worse
Disjoint Alphabet	0.3588	0.3360	1.0677	0.9999	0.9999	1.07×	SAM
Mutated Implant	0.4070	0.2836	1.4349	1.0000	1.0000	1.43×	SAM
Near Identical	0.3404	0.3077	1.1061	0.9999	0.9999	1.11×	SAM
Random Uniform	0.3387	0.3556	0.9524	0.9998	0.9999	1.05×	ESA
Repetitive With Noise	0.4468	0.2824	1.5823	1.0000	1.0000	1.58×	SAM

AI Declaration

The usage of AI tools within this project is documented here, including the prompts used and the outputs received.

System	Prompt	Usage
GitHub Copilot 1	Asked to delete the existing project and provide a simple LaTeX template with a shared preamble, a Chapters subdirectory, TOC, glossary, abbreviations, and Roman numerals for non-content pages.	Template structure and LaTeX setup provided
GitHub Copilot 2	Requested additional title page lines (WAB header, reviewer, module) for the main document.	Title page metadata and layout updated
GitHub Copilot 3	Requested uppercase Roman numerals for front matter and lowercase Roman numerals for back matter page numbering.	Page numbering adjusted in main and exposee
GitHub Copilot 4	Requested exposee title page to include shared info fields (WAB header, department, module, reviewer).	Exposee title page updated to include shared metadata
GitHub Copilot 5	Requested adding the provadis-hochschule.pdf logo to the top right corner of both main and exposee title pages.	Logo added to top right of both title pages
GitHub Copilot 6	Reported that Glossary and Abbreviations sections were missing from compiled output.	Added example `\newacronym` entries to abbreviations.tex
GitHub Copilot 7	Reported that glossary section still not appearing in final PDF.	Updated settings.tex to include `automake` option in glossaries package to enable automatic glossary generation
GitHub Copilot 8	Requested Python benchmark code for LCS comparison with multiple synthetic scenarios (random strings, implanted mutated substrings, etc.)	Created benchmark script and documentation in Code folder
GitHub Copilot 9	Asked to add a progress-bar-like output.	Added optional single-line in-place progress output with percentage, completed/total steps, elapsed time, ETA, and `-progress`/`-no-progress` flags

System	Prompt	Usage
GitHub Copilot 10	Reported that ETA was inaccurate due to varying string lengths	Reworked ETA estimation to be algorithm- and length-aware using observed per-bucket runtimes
GitHub Copilot 11	Requested separate build and query timing (and plots).	Refactored benchmark to record build/query time separately and added total time summaries
GitHub Copilot 12	Requested plots using statistics beyond the mean (median, standard deviation, etc.).	Expanded summary stats and generated mean+std and median+IQR plots alongside memory plots
GitHub Copilot 13	Requested splitting benchmarking and plotting so plot generation can run separately, with a shared CLI entry point and default run-then-plot behavior.	Added CLI mode switch (run/plot/both) to reuse saved CSV results for plotting without rerunning benchmarks
GitHub Copilot 14	Requested an error if the two algorithms return different LCS result strings for the same input.	Added strict substring equality check (in addition to length) and raise an error on mismatch
GitHub Copilot 15	Requested that, in case of multiple valid LCS results, both algorithms return all results and compare the sets.	Updated both algorithms to return all max-length substrings and compare result sets for correctness
GitHub Copilot 16	Requested refactoring the benchmark script into helper modules under a helpers subdirectory, with separate algorithm imports and plotting helper, and clean top-level imports in the main file.	Split benchmark code into dedicated helper modules (SAM, ESA, plotting, generation, statistics, benchmarking) and simplified main script orchestration
GitHub Copilot 17	Requested mean, median, standard deviation, and IQR for memory usage plots as well.	Added memory quartile aggregation and generated memory mean+SD plus median+IQR plots
GitHub Copilot 18	Asked for a practical memory/space-complexity metric and whether build-time and query-time memory can be measured separately.	Added separate build/query phase memory metrics and a persistent index-size metric, with aggregation and plots for comparison
GitHub Copilot 19	Asked whether adding total time (build + query) columns and graphs with full statistics would make sense.	Added full total-time statistics (mean, median, std, IQR, Q1, Q3) to summaries and created dedicated total-time plots
GitHub Copilot 20	Requested moving plot legends away from the top center because they obscured titles.	Repositioned all figure legends to the right side outside the plotting area to keep titles unobstructed
GitHub Copilot 21	Requested an additional output file containing both generated strings and their LCS value(s).	Added export of one CSV per generated case with ‘s‘, ‘t‘, LCS length, and all LCS substrings

System	Prompt	Usage
GitHub Copilot 22	Asked to replace the lambda with a tuple-key-based approach.	Replaced lambda-based sorting with a tuple-key-based approach for suffix array construction, improving performance
GitHub Copilot 23	Asked to refactor the plotting code to reduce repetition and make it more modular.	Refactored plotting code to use a single generic plotting function
GitHub Copilot 24	Asked to replace the suffix array construction with the SA-IS algorithm.	Replaced doubling-based suffix array construction with an SA-IS implementation (linear-time induced sorting with LMS recursion), keeping the same public API
GitHub Copilot 25	Reported that generated benchmark tables do not fit on the page and requested a smart code-side solution (splitting tables if needed) instead of shrinking them.	Updated the table-export generator to split each algorithm summary into multiple narrower tables (timing, memory, index size)
GitHub Copilot 26	Requested that the generated benchmark tables should not show up in the List of Tables.	Adjusted the generator to avoid captions and instead use a custom numbered title with `\refstepcounter{table}` and `\label` for cross-references
GitHub Copilot 27	Requested that the generated tables are still slightly too large and should use a smaller font size.	Wrapped generated tabular blocks in `\small` within the generator to reduce width without making tables unreadably tiny
GitHub Copilot 28	Asked to calculate per-metric, per-scenario linear regression slope ratios (SAM slope / ESA slope) and export them as LaTeX tables and a CSV, so the hardware-independent ratios can be cited in the report text.	Added `export_slope_ratio_tables` to `statistics_io.py` using `numpy.polyfit` on median values vs. string length
GitHub Copilot 29	Asked to calculate the R squared values for the linear regressions and include them in the slope ratio tables.	Updated `export_slope_ratio_tables` to also calculate R^2 using `numpy.polyval` and include it in the output tables

Declaration of Authorship

I hereby confirm that I have personally and independently prepared the present work and have not used any sources or aids other than those specified. All passages taken verbatim or in substance from other sources are identified as such. The drawings, illustrations and tables in this work are created by me or have been provided with an appropriate source reference. This work has not been submitted by me to any other university in the same or similar form for the acquisition of an academic degree.

Frankfurt, March 9, 2026 .....

Rubin Chempananickal James